Software Testing & CI/CD

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

27 March 2026

Today's Agenda

Agenda

Recap: Containerisation & Docker

Last week, Prof. Marc Hanheide introduced us to the reality of deploying software in the real world using **Containerisation**.

- **The Problem:** ”It works on my machine!”
- **The Solution:** Docker containers package the application code along with all its dependencies, libraries, and OS configurations into a single, reproducible image.
- **Microservices:** Using `docker-compose` to orchestrate multiple independent services (e.g., connecting your Frontend, Backend, and the Virtual Robot Simulator).

Today, we look at how to leverage those predictable containers to automate our Software Testing!

Today's Learning Objectives

By the end of this lecture, you should be able to:

1. **Understand** the critical importance of software testing in mitigating real-world deployment risks.
2. **Differentiate** between the various levels of testing (Unit, Integration, System, Acceptance).
3. **Distinguish** between functional and non-functional testing methodologies.
4. **Explain** the concept of the Test Automation Pyramid.
5. **Integrate** automated testing into a Continuous Integration / Continuous Deployment (CI/CD) pipeline.

Agenda

Why do we test?

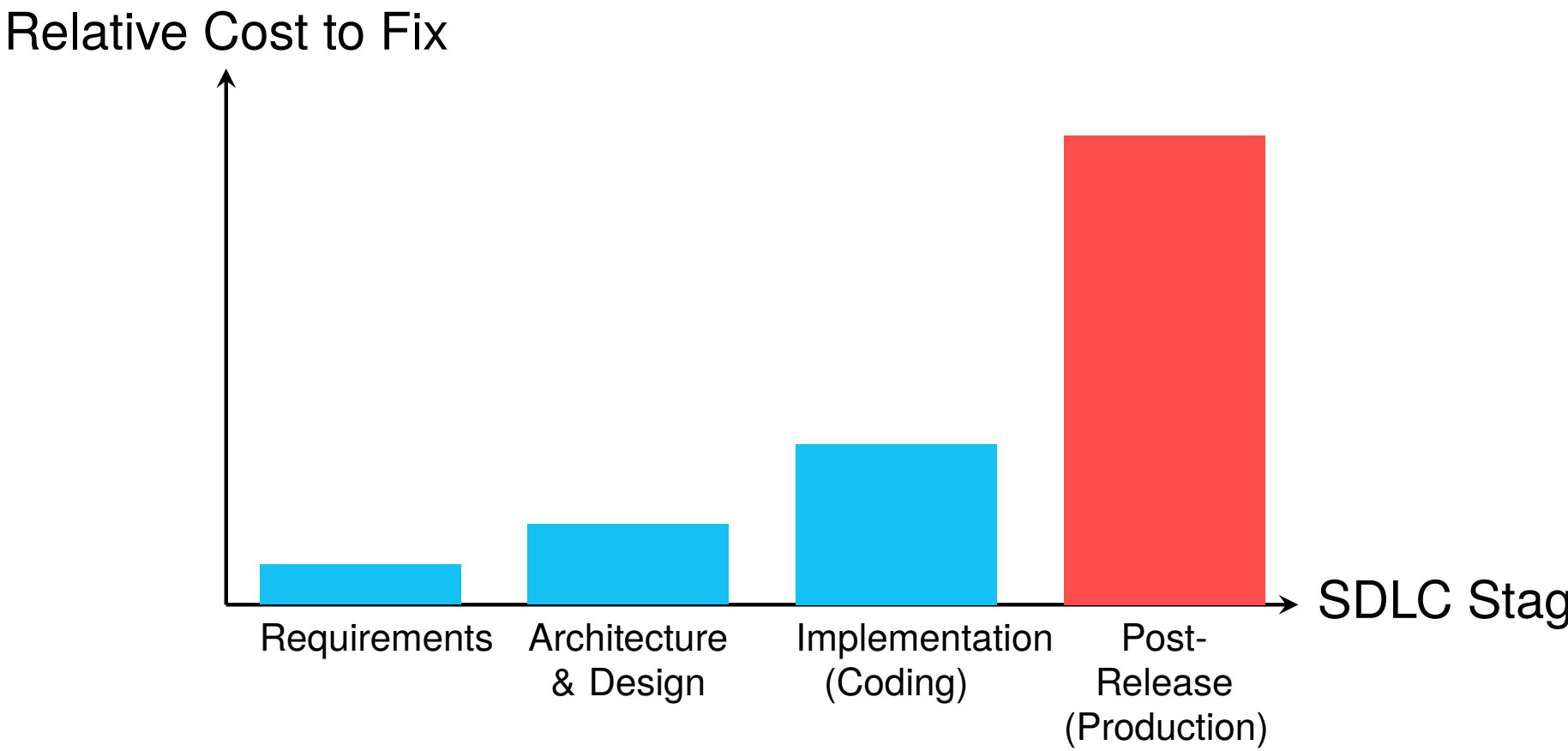
Software bugs are inevitable, but shipping them to production is a choice. Testing ensures software quality, reliability, and security.

Real-World Failures:

- **Public Embarrassment:** AI demos failing live on stage (e.g., Google's Gemini launch anomalies, LG's robot refusing to respond at CES 2018).
- **Financial Ruin:** The Ariane 5 rocket explosion ($370 million lost due to a simple 64-bit to 16-bit integer overflow bug).
- **Safety Risks:** In robotics (like your Ground Control Station), an untested "Move" command could cause physical damage to the hardware or environment.

The Cost of Fixing Defects

The later a bug is found in the Software Development Life Cycle (SDLC), the more expensive it is to fix.



Agenda

Testing Levels (The V-Model)

Testing is not a single phase; it occurs at multiple levels, mapping directly to how the software was designed.

1. **Unit Testing:** Tests individual components or functions in isolation. Usually written by the developers themselves.
2. **Integration Testing:** Tests how different modules or services work when combined (e.g., Does the Python backend correctly parse JSON from the Robot API?).
3. **System Testing:** Evaluates the complete, fully integrated software product to ensure it meets technical specifications.
4. **Acceptance Testing:** Conducted by the end-users or clients to verify that the system satisfies the business requirements (Does it actually solve the problem?).

Zooming in: Unit Testing

Unit tests check the smallest testable parts of an application. They should be fast, isolated, and deterministic.

Example (Python using `pytest`):

```python
# core_logic.py
def is_valid_coordinate(x, y):
    if x < 0 or y < 0:
        return False
    return True

# test_core_logic.py
def test_is_valid_coordinate():
    # Arrange & Act
    valid = is_valid_coordinate(5, 10)
    invalid = is_valid_coordinate(-1, 5)

    # Assert
    assert valid == True
    assert invalid == False
```

If another developer later breaks the `is_valid_coordinate` logic, this test will

Agenda

Testing Types: Functional vs. Non-Functional

We divide testing into *what* the system does versus *how well* it does it.

Functional Testing

Validates the software against functional requirements.

- **Examples:** Can the user log in? Does the ”Emergency Stop” button halt the robot? Does the database save the audit log?
- **Smoke Testing:** A subset of functional tests to check if the crucial functions work before doing a deeper test pass.

Non-Functional Testing

Evaluates the operational aspects of the system.

- **Performance Testing:** How does the backend handle 1,000 simultaneous robot status requests?

Regression Testing

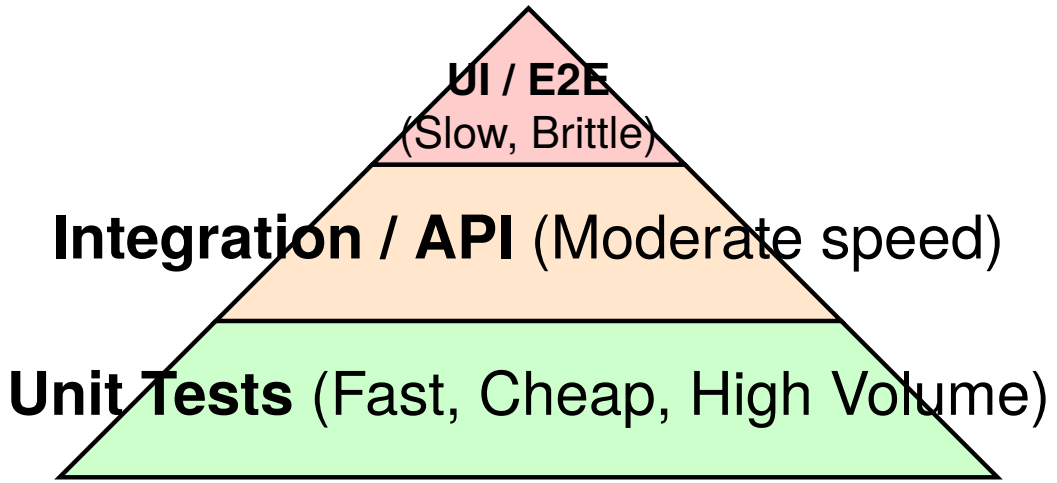
Definition: Re-running functional and non-functional tests to ensure that previously developed and tested software still performs after a change.

- As your codebase grows, adding a new feature often accidentally breaks an old one (a ”regression”).
- Manual regression testing is incredibly tedious and error-prone.
- **Solution:** Automated test suites. Every time you commit code to GitHub, the automated runner executes the entire regression suite in seconds.

Agenda

The Test Automation Pyramid

How should we distribute our automated testing efforts?



- Write **lots** of Unit tests (they execute in milliseconds).
- Write **some** Integration tests (they require databases/containers to spin up).
- Write **few** End-to-End UI tests (they require simulating a real browser, which is slow and prone to breaking on minor visual changes).

Bringing it all together: CI / CD

Continuous Integration (CI): The practice of frequently merging code changes into a central repository, where automated builds and tests are run.
Continuous Deployment (CD): Automating the release of validated code to a production environment.

How it works in your project (GitHub Actions):

1. You push code to GitHub.
2. GitHub spins up an isolated server.
3. It builds your **Docker Containers** (what you learned last week!).
4. It runs your **Automated Test Suite** (what you learned today!).
5. If tests pass, it merges the code. If tests fail, it blocks the merge.

Agenda

This Week's Lab: Writing Tests & CI

In the workshop, you will implement automated testing for your Assessment 1 Robot Management System.

- **Task 1: Unit Testing:** Write isolated tests for your backend logic (e.g., verifying RBAC permissions or coordinate math).
- **Task 2: API Testing:** Write tests to verify your backend routes return the correct JSON HTTP status codes.
- **Task 3: GitHub Actions CI:** Create a YAML pipeline to automatically run your tests every time you push to your repository.

Reading & Resources

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapters 8 (Testing) and 25 (Configuration Management).
- **PyTest Documentation:** Excellent starting point if you are writing a Python backend.
- **Jest Documentation:** The industry standard if you are writing a Node.js/JavaScript backend.
- **GitHub Actions Docs:** Review the syntax for creating CI workflows.

Q & A

Any Questions?

fdelduchetto@lincoln.ac.uk